

文档说明

本文档用途：本文档以"课程信息查询"功能为例，完整演示如何从需求出发，通过自顶向下、逐步求精的方式，逐层展开得到可执行代码的全过程。学生可参照此示例，完成自己系统中其他功能模块的设计文档编写。

阅读顺序：请按章节编号顺序阅读。每个阶段之间设有过渡说明，解释本阶段是在前一阶段的基础上如何进行细节展开的。

本文档结构：

表 1 文档结构说明

章节	内容	作用
一、用例规约	功能的参与者、流程、业务规则	定义需求基线
二、用例活动图	学生与系统的交互泳道图	可视化业务流程
三、系统活动图	学生、前端、后端三泳道协作图	注入技术架构分层
四、Web后端顺序图	类职责表、类图、顺序图、流程说明	精确设计对象交互
五、后端代码	Controller / Service / Repository 四层代码	设计落地为程序
六、过程正文说明	对系统活动图每一步的文字解读	形成需求到代码的闭环

编写建议：学生在撰写自己的功能设计文档时，应遵循与本文档相同的章节结构和求精路线——先写需求规约，再画业务流程图，再注入技术分层，再细化到类和方法，最后给出代码实现。各阶段之间的类名、方法名应严格保持一致。

一、用例规约

项目	内容
用例名	课程信息查询
简要描述	学生通过该用例查询指定课程的详细信息
参与者	学生
涉众	学生：快速、准确地获取课程的详细信息
相关用例	无
前置条件	学生已进入课程信息查询页面
后置条件	如果查询成功，系统显示该课程信息；如果课程不存在，系统给出相应的提示信息
基本事件流	(1) 用例起始于学生需要查询某门课程的信息 (2) 系统显示课程查询页面 (3) 学生输入课程编号（B-1） (4) 系统根据课程编号查询课程信息（D-2） (5) 系统显示课程信息，用例结束
补充约束-数据需求	D-1 课程信息：课程编号、课程名称、学分、授课教师、课程描述等
补充约束-业务规则	B-1 课程编号必须为大于0的正整数

编写说明：用例规约表以结构化方式概述了功能的参与者、触发条件、执行流程及业务规则，为后续活动图和代码设计提供需求基线。

二、用例活动图

(这部分内容在论文中没有，这里显示出来是为了后续设计的展开，方便大家理解)

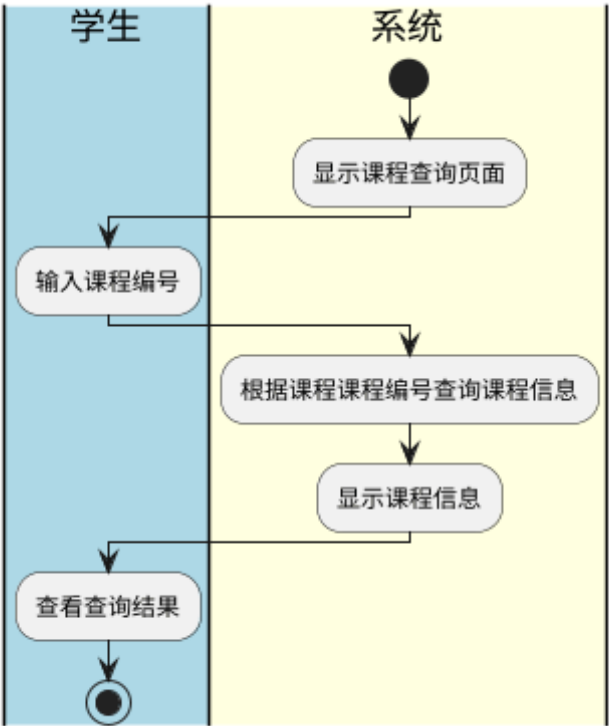


图 1 用例活动图——学生与系统的交互泳道图

编写说明：用例活动图以两条泳道（学生、系统）展示该用例的交互流程。系统侧包含三个核心处理——"显示课程查询页面""根据课程编号查询课程信息""显示课程信息"，对应用例规约基本事件流的步骤（2）（4）（5），流程起始于系统显示课程查询页面。

用例活动图从业务视角描述了交互流程，但其中的"系统"仍是一个黑盒。进入系统活动图阶段，我们将这个笼统的"系统"泳道细分为"web前端"与"Web后端"两个技术层，在保持原有流程骨架不变的前提下，逐步注入架构细节，实现从业务描述到技术设计的第一次求精。

三、系统活动图

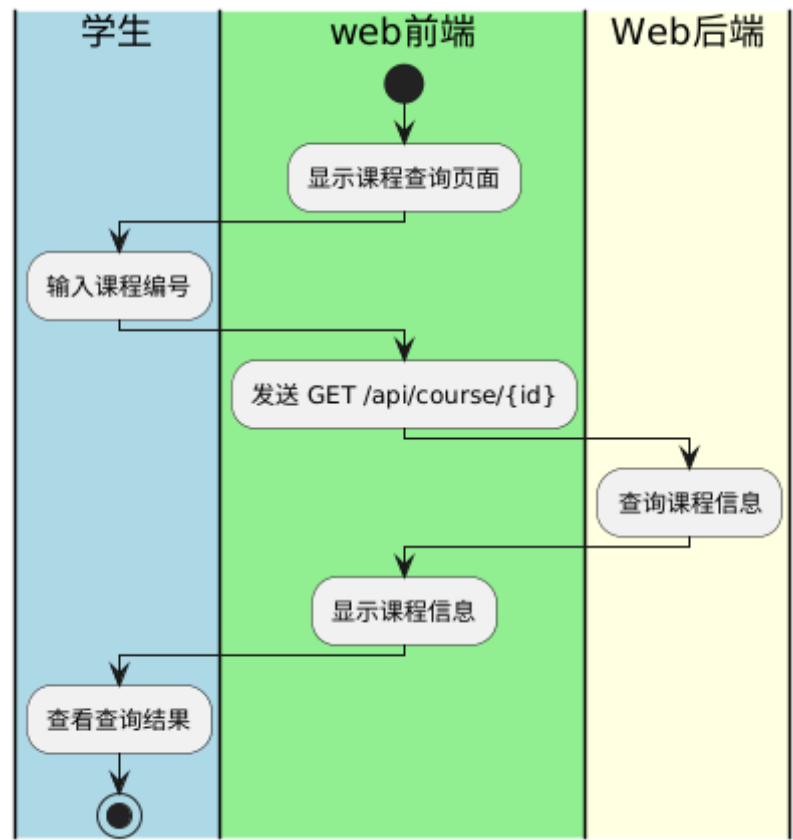


图 2 系统活动图——学生、web前端、Web后端三泳道协作图

编写说明：系统活动图（泳道图）依据用例活动图展开，展示该功能在"学生—web前端—Web 后端"三个主体之间的协作过程。与用例活动图保持一致的流程结构，仅将"系统"泳道细分为"web前端"和"Web后端"两个技术层，体现数据在三层之间的流转路径。

系统活动图展示了数据在三层之间的流转路径，但每一层内部的处理逻辑仍是一个黑盒。Web后端顺序图则进一步聚焦于Web后端这一泳道，将其内部协作细化为Controller、Service、Repository等具体类的职责划分及其方法调用，通过静态类图和动态顺序图精确描述对象间的交互关系，完成从架构分层到类级设计的逐层分解。

4.1 Web后端课程信息查询功能类

表 2 Web后端课程信息查询功能类职责表

类名	描述
CourseController	课程信息服务 Web 开放接口类（Controller 层），负责接收前端 HTTP 请求并返回 JSON 响应。
CourseService	课程信息业务逻辑服务接口，定义课程模块对外暴露的公共业务操作方法。
CourseServiceImpl	课程信息业务逻辑服务实现类，实现 CourseService 接口，封装课程查询等核心逻辑。
CourseRepository	课程信息数据访问接口（Repository / DAO 层），继承 JpaRepository，由 Spring Data JPA 自动生成数据库 CRUD 实现。
Course	课程信息实体类，用于在各层之间传递数据，对应数据库中的课程表。

编写说明：功能类表的作用是**提前告知读者：**Web 后端为了实现该功能，需要引入哪些核心类、每个类承担什么职责。这张表相当于后文类图和顺序图的"索引"，让读者在查看静态结构与动态交互之前，先对各参与类的分工有初步了解。学生填写时应注意"类名"与后文代码及类图严格保持一致，"描述"用一句话概括该类在本功能中的核心职责即可，避免写成通用的技术说明。

4.2 Web后端课程信息查询功能类类图

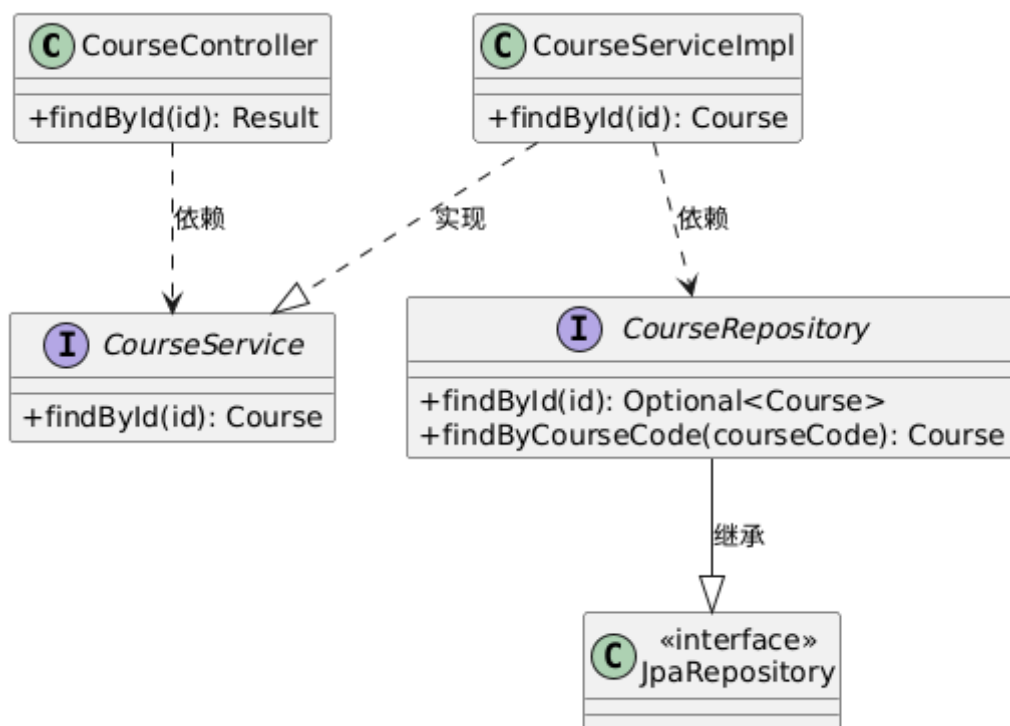


图 3 Web后端课程信息查询功能类类图

编写说明：类图的作用是站在**静态结构视角**，展示实现该功能所需的类、接口及其相互关系（依赖、实现、继承）。读者通过类图可以快速把握系统的分层架构：Controller 依赖 Service 接口、Service 实现类实现 Service 接口并依赖 Repository、Repository 继承框架提供的 JpaRepository。学生在绘制时应遵循“只列类名和 public 方法”的原则，隐藏内部私有属性与复杂方法，突出类之间的关系线条，确保图面简洁、关系清晰。

4.3 Web后端课程信息查询流程的设计

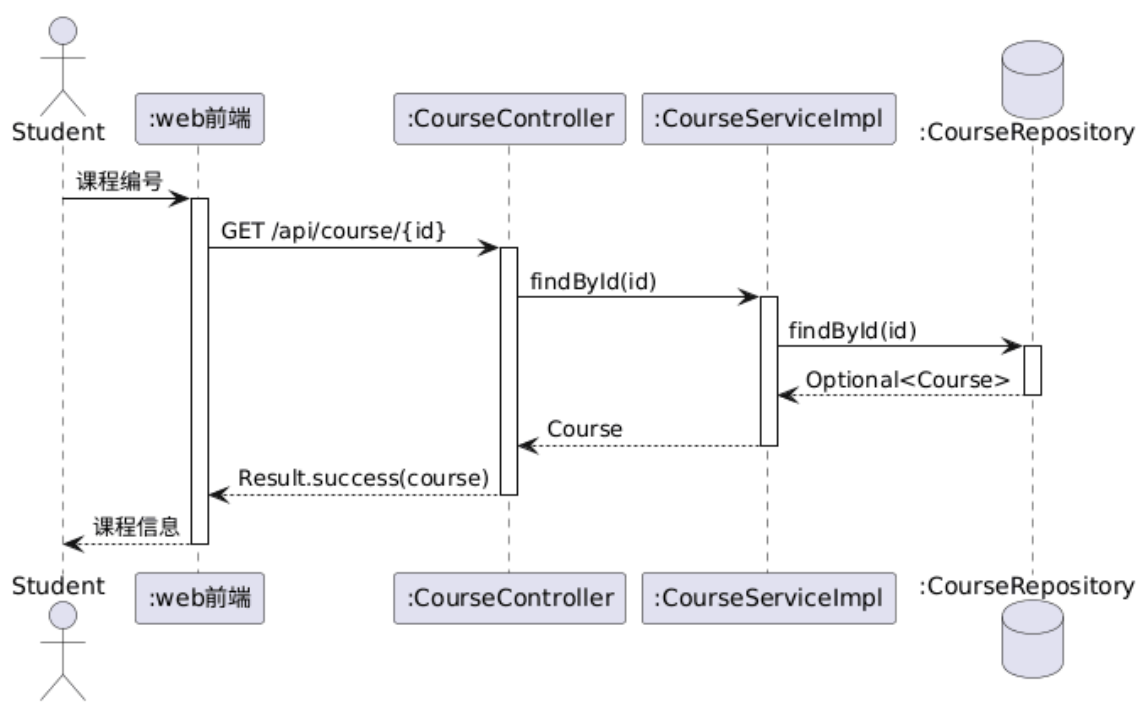


图 4 Web后端课程信息查询流程顺序图

编写说明：顺序图（时序图）的作用是站在**动态运行视角**，按时间顺序展示对象之间的消息交互。与类图的"静态结构"互补，顺序图回答的是"代码跑起来时，谁先调用谁、传什么参数、返回什么结果"。学生在绘制时必须保留**生命线（Lifeline）**，并使用实线箭头表示同步调用、虚线箭头表示返回结果；参与者的排列顺序建议与代码分层一致（web前端 → Controller → Service → Repository），使阅读方向自然流畅。

4.4 Web后端课程信息查询流程的正文说明

Web 后端课程信息查询流程说明如下：

- （1）学生在前端页面输入需要查询的课程ID，点击【查询】按钮，web前端将课程ID作为URL路径参数，通过 GET 请求发送至 Web 后端的 `CourseController`。
- （2）`CourseController` 的 `findById()` 方法接收请求，利用 Spring MVC 的 `@PathVariable` 注解从URL路径中提取课程ID参数，随后调用 `CourseService` 接口的 `findById()` 方法进入业务逻辑层。
- （3）`CourseServiceImpl` 实现类执行具体的业务逻辑：调用 `CourseRepository` 接口的 `findById()` 方法，由 Spring Data JPA 自动生成的数据访问实现去数据库中查询对应ID的课程记录。查询结果以 `Optional<Course>` 形式返回给服务层，服务层取出其中的课程对象后返回给 Controller。

(4) `CourseController` 使用统一的 `Result` 工具类封装查询结果，以 JSON 格式回传给 web 前端，web 前端解析响应数据后向学生展示课程的详细信息（如课程名称、学分、授课教师等）。

编写说明：Web 后端流程正文说明的目的是对顺序图中的**每一次消息交互进行细化解读**，把"对象之间传了什么消息"翻译成"业务上做了什么处理、调用了什么方法、数据状态发生了什么变化"。这部分是连接"设计图"与"代码实现"的关键桥梁：读者对照顺序图和这段文字，就能明白代码中每一行对应的设计意图。学生在编写时应**逐编号对应顺序图中的消息序号**，使用"调用…方法"、"返回…对象"、"更新…属性"等精确动词，避免空泛的"系统进行处理"之类描述。

五、后端代码

以下给出 Web 后端实现"课程信息查询"功能所涉及的 **4 个核心文件**（1 个 Controller、1 个 Service 接口、1 个 Service 实现类、1 个 Repository 接口）。为聚焦设计思路，功能中需要用到但非本示例重点的其他类（如统一响应封装类、课程实体类等），均以注释方式说明其用途，不再额外展开。

5.1 CourseController.java (Controller 层)

```
package com.example.demo.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.example.demo.common.Result;
import com.example.demo.entity.Course;
import com.example.demo.service.CourseService;

/**
 * 课程信息Controller类 (Web开放接口层)
 * 负责接收前端HTTP请求，调用Service层完成业务处理，返回统一JSON响应
 */
@RestController
@RequestMapping("/api/course")
public class CourseController {

    @Autowired
    private CourseService courseService;

    /**
     * 根据课程ID查询课程信息
     * @param id 课程ID (从URL路径中获取)
     * @return 统一响应结果，包含查询到的课程信息
     */
    @GetMapping("/{id}")
    public Result findById(@PathVariable Long id) {
        // 调用业务逻辑层查询课程信息
        Course course = courseService.findById(id);
        // 封装成功响应返回前端
        return Result.success(course);
    }
}
```

5.2 CourseService.java (Service 接口层)

```
package com.example.demo.service;

import com.example.demo.entity.Course;

/**
 * 课程信息业务逻辑服务接口
 * 定义课程模块对外暴露的公共业务能力
 */
public interface CourseService {

    /**
     * 根据课程ID查询课程信息
     * @param id 课程ID
     * @return 课程信息对象，若不存在则返回null
     */
    Course findById(Long id);
}
```

5.3 CourseServiceImpl.java (Service 实现层)

```
package com.example.demo.service.impl;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.example.demo.entity.Course;
import com.example.demo.repository.CourseRepository;
import com.example.demo.service.CourseService;

/**
 * 课程信息业务逻辑服务实现类
 * 实现CourseService接口，封装课程查询相关的核心业务逻辑
 */
@Service
public class CourseServiceImpl implements CourseService {

    @Autowired
    private CourseRepository courseRepository;

    @Override
    public Course findById(Long id) {
        // (1) 调用数据访问层，根据课程ID查询数据库
        // 返回查询到的课程实体对象，若不存在则返回null
        return courseRepository.findById(id).orElse(null);
    }
}
```

5.4 CourseRepository.java (Repository / DAO 层)

```
package com.example.demo.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.example.demo.entity.Course;

/**
 * 课程信息数据访问接口 (Repository / DAO层)
 * 继承JpaRepository, 由Spring Data JPA自动生成基本的增删改查实现
 */
@Repository
public interface CourseRepository extends JpaRepository<Course, Long> {

    /**
     * 根据课程编号查询课程信息
     * 方法命名符合Spring Data JPA规范, 框架会自动生成查询实现
     * @param courseCode 课程编号 (如: CS101)
     * @return 课程信息对象, 若不存在则返回null
     */
    Course findByCourseCode(String courseCode);
}
```

编写说明：代码展示应遵循**分层清晰、注释到位、重点突出**的原则。Controller 层体现接口定义与请求映射，Service 接口层体现面向接口编程的设计思想，Service 实现层体现核心业务逻辑，Repository 层体现数据访问的简化与框架支持。学生在提供代码时，应确保类名、方法名、包名与前面类图、顺序图完全一致，形成"需求—设计—代码"的完整闭环。

六、课程信息查询过程的正文说明

课程信息查询过程说明如下：

- (1) 学生进入课程查询页面，输入需要查询的课程ID，点击【查询】按钮。
- (2) web前端收集学生输入的课程ID，将其作为URL路径参数，通过 HTTP GET 请求发送至 Web 后端的 `/api/course/{id}` 接口。
- (3) Web 后端接收请求后执行课程信息查询处理，将查询结果以 JSON 格式返回给 web 前端，web前端解析后向学生展示课程详细信息。Web 后端内部的具体处理流程见第四章顺序图。

编写说明：过程正文说明的目的是对泳道图中的每一步进行**文字级展开解释**，把"图里发生了什么"翻译成"系统具体做了什么、数据如何传递"。这部分文字起到承上启下的作用：向上对应泳道图的整体流程，向下为后文类设计和顺序图的细节提供铺垫。学生编写时应逐条对应图中的活动节点，使用"谁做了什么、产生了什么数据、发送给谁"的句式，避免笼统描述。